

BasePruningMethod#

<https://pytorch.org/docs/stable/generated/torch.nn.utils.prune.BasePruningMethod>

Description:

Abstract base class for creation of new pruning techniques. Provides a skeleton for customization requiring the overriding of methods such as `compute_mask()` and `apply()`. Add pruning on the fly and reparametrization of a tensor. Adds the forward pre-hook that enables pruning on the fly and the reparametrization of a tensor in terms of the original tensor and the pruning mask. module (nn.Module) – module containing the tensor to prune

Function Signature

```
class torch.nn.utils.prune.BasePruningMethod[source]#  
classmethod apply(module, name, *args,  
importance_scores=None, **kwargs)[source]#
```

Parameters

```
apply_mask(module)[source]#
```

Parameters

Returns:

pruned version of tensor t.

Notes & Warnings

NOTE

Note Pruning itself is NOT undone or reversed!

Detailed Documentation

Documentation

Abstract base class for creation of new pruning techniques.

Provides a skeleton for customization requiring the overriding of methods such as `compute_mask()` and `apply()`.

Add pruning on the fly and reparametrization of a tensor.

Adds the forward pre-hook that enables pruning on the fly and the reparametrization of a tensor in terms of the original tensor and the pruning mask.

module (nn.Module) – module containing the tensor to prune

name (str) – parameter name within module on which pruning will act.

args – arguments passed on to a subclass of `BasePruningMethod`

importance_scores (torch.Tensor) – tensor of importance scores (of same shape as module parameter) used to compute mask for pruning. The values in this tensor indicate the importance of the corresponding elements in the parameter being pruned. If unspecified or None, the parameter will be used in its place.

kwargs – keyword arguments passed on to a subclass of a BasePruningMethod

Simply handles the multiplication between the parameter being pruned and the generated mask.

Fetches the mask and the original tensor from the module and returns the pruned version of the tensor.

module (nn.Module) – module containing the tensor to prune

pruned version of the input tensor

pruned_tensor (torch.Tensor)

Compute and returns a mask for the input tensor t.

Starting from a base default_mask (which should be a mask of ones if the tensor has not been pruned yet), generate a random mask to apply on top of the default_mask according to the specific pruning method recipe.

t (torch.Tensor) – tensor representing the importance scores of the

prune. (parameter to) –

default_mask (torch.Tensor) – Base mask from previous pruning

is (that need to be respected after the new mask) –

t. (applied. Same dims as) –

mask to apply to t, of same dims as t

Compute and returns a pruned version of input tensor t.

According to the pruning rule specified in `compute_mask()`.

t (`torch.Tensor`) – tensor to prune (of same dimensions as `default_mask`).

importance_scores (`torch.Tensor`) – tensor of importance scores (of same shape as t) used to compute mask for pruning t. The values in this tensor indicate the importance of the corresponding elements in the t that is being pruned. If unspecified or `None`, the tensor t will be used in its place.

default_mask (`torch.Tensor`, optional) – mask from previous pruning iteration, if any. To be considered when determining what portion of the tensor that pruning should act on. If `None`, default to a mask of ones.

pruned version of tensor t.

Remove the pruning reparameterization from a module.

The pruned parameter named name remains permanently pruned, and the parameter named name+'_orig' is removed from the parameter list. Similarly, the buffer named name+'_mask' is removed from the buffers.

Pruning itself is NOT undone or reversed!